

Fundamentos da POO – Visibilidade

UNIVERSIDADE DO OESTE DE SANTA CATARINA - UNOESC

Ciência da Computação | Programação II

Prof.: Leandro Otavio Cordova Vieira



Objetivos da Aula



Compreender Visibilidade

Entender os conceitos fundamentais de visibilidade no PHP e sua importância no desenvolvimento orientado a objetos.



Aplicar em Exemplos

Ver exemplos práticos de implementação dos diferentes níveis de visibilidade em cenários reais de programação.



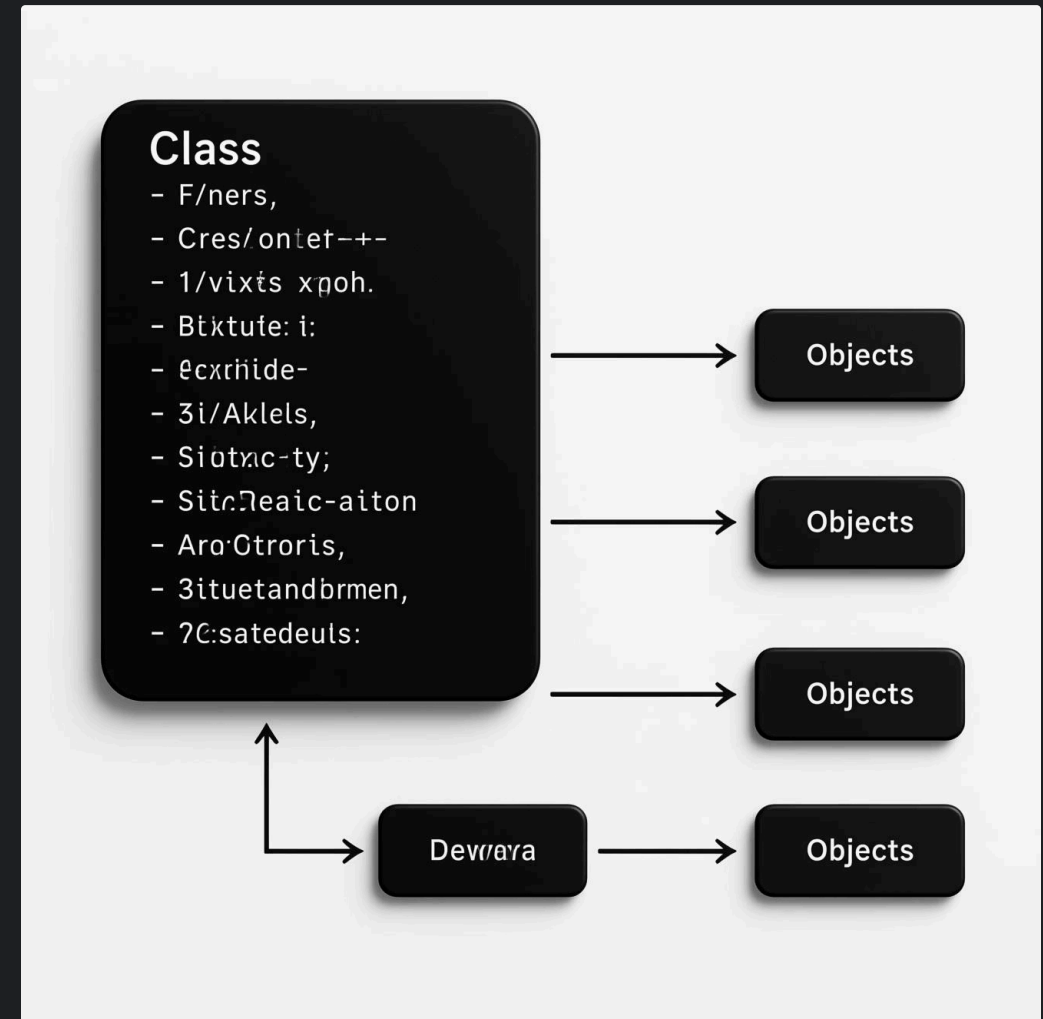
Resolver Atividades

Trabalhar na lista de atividades A1/1 para consolidar o conhecimento adquirido durante a aula.

Relembrando POO

Conceitos Fundamentais

- **Classe:** Modelo que define atributos e comportamentos
- **Objeto:** Instância concreta de uma classe
- **Atributos:** Características/propriedades do objeto
- **Métodos:** Ações/comportamentos que o objeto pode realizar



O **encapsulamento** é um dos pilares fundamentais da POO, permitindo ocultar detalhes internos e proteger os dados



O que é Visibilidade?

A visibilidade em POO define o **nível de acesso** que outras classes e objetos têm aos atributos e métodos de uma classe.

Controle de Acesso

Define regras específicas sobre quem pode acessar cada componente de uma classe

Proteção de Dados

Protege informações sensíveis e implementações internas da classe

Organização do Código

Estabelece interfaces claras para interação entre diferentes partes do sistema

No PHP, existem três níveis principais de visibilidade: **public**, **private** e **protected**.

Visibilidade: public

Características:

- Acesso **completamente irrestrito** aos atributos e métodos
- Elementos podem ser acessados de **qualquer lugar** do código
- Visíveis dentro da classe, em classes herdeiras e fora da classe
- Declarados com a palavra-chave `public`

Quando usar:

- Para métodos que devem ser chamados por outros objetos
- Interfaces públicas da sua classe
- Funcionalidades que devem ser amplamente acessíveis



```
class Calculadora {  
    public $versao = "1.0";  
  
    public function somar($a, $b) {  
        return $a + $b;  
    }  
}
```

Exemplo de public

```
class Usuario {  
    public $nome;  
    public $email;  
  
    public function __construct($nome, $email) {  
        $this->nome = $nome;  
        $this->email = $email;  
    }  
  
    public function exibirPerfil() {  
        return "Nome: " . $this->nome .  
            "\nEmail: " . $this->email;  
    }  
}  
  
// Utilizando a classe  
$usuario = new Usuario("Maria Silva", "maria@email.com");  
  
// Acesso direto aos atributos públicos  
echo $usuario->nome; // "Maria Silva"  
  
// Acesso ao método público  
echo $usuario->exibirPerfil();
```

Neste exemplo, qualquer parte do código pode **acessar** e **modificar** diretamente os atributos `$nome` e `$email`, além de poder chamar o método `exibirPerfil()`.

Visibilidade: private

Características:

- Acesso **restrito apenas à própria classe**
- Elementos **não podem** ser acessados de fora da classe
- Elementos **não podem** ser acessados por classes herdeiras
- Declarados com a palavra-chave `private`

Quando usar:

- Para proteger dados sensíveis
- Em implementações internas que não devem ser expostas
- Quando a lógica de validação precisa ser centralizada
- Para reduzir dependências e evitar uso incorreto



```
class Pessoa {  
  private $cpf;  
  
  private function validarCPF($cpf) {  
    // Lógica de validação  
  }  
}
```

Exemplo de private

```
class ContaBancaria {
    private $saldo;
    private $senha;

    public function __construct($saldoInicial, $senha) {
        $this->saldo = $saldoInicial;
        $this->senha = $senha;
    }

    public function sacar($valor, $senhaInformada) {
        if ($this->validarSenha($senhaInformada)) {
            if ($valor <= $this->saldo) {
                $this->saldo -= $valor;
                return "Saque de R$ {$valor} realizado com sucesso!";
            } else {
                return "Saldo insuficiente!";
            }
        }
        return "Senha incorreta!";
    }

    private function validarSenha($senhaInformada) {
        return $this->senha === $senhaInformada;
    }
}

$conta = new ContaBancaria(1000, "1234");

// ERRO: Não é possível acessar atributo privado
// echo $conta->saldo;

// ERRO: Não é possível chamar método privado
// $conta->validarSenha("1234");

// Correto: usar o método público que acessa internamente os atributos privados
echo $conta->sacar(500, "1234");
```


Visibilidade: protected

Características:

- Acesso permitido na **própria classe** e em **classes herdeiras**
- Elementos **não podem** ser acessados fora da hierarquia de classes
- Nível intermediário entre `public` e `private`
- Declarados com a palavra-chave `protected`

Quando usar:

- Para funcionalidades que devem ser compartilhadas apenas com subclasses
- Quando se trabalha com herança e polimorfismo
- Para implementações que devem ser extensíveis, mas não diretamente acessíveis



```
class Animal {  
    protected $idade;  
  
    protected function comer() {  
        // Código comum  
    }  
}
```

Exemplo de protected

```
class Funcionario {
    protected $salarioBase;
    protected $nome;

    public function __construct($nome, $salarioBase) {
        $this->nome = $nome;
        $this->salarioBase = $salarioBase;
    }

    protected function calcularBonificacao() {
        return $this->salarioBase * 0.1;
    }

    public function getSalarioTotal() {
        return $this->salarioBase + $this->calcularBonificacao();
    }
}

class Gerente extends Funcionario {
    private $equipe;

    public function __construct($nome, $salarioBase, $equipe) {
        parent::__construct($nome, $salarioBase);
        $this->equipe = $equipe;
    }

    // Sobrescrita do método protegido
    protected function calcularBonificacao() {
        // Acesso ao atributo protegido da classe pai
        return $this->salarioBase * 0.2;
    }
}

$funcionario = new Funcionario("João", 2000);
$gerente = new Gerente("Maria", 4000, 5);

// ERRO: Não é possível acessar atributo protegido diretamente
// echo $funcionario->salarioBase;

// ERRO: Não é possível chamar método protegido diretamente
// $funcionario->calcularBonificacao();

// Correto: acessar através de métodos públicos
echo $funcionario->getSalarioTotal(); // 2200
echo $gerente->getSalarioTotal(); // 4800
```

Cenário Real: Sistema de Usuários

Problema:

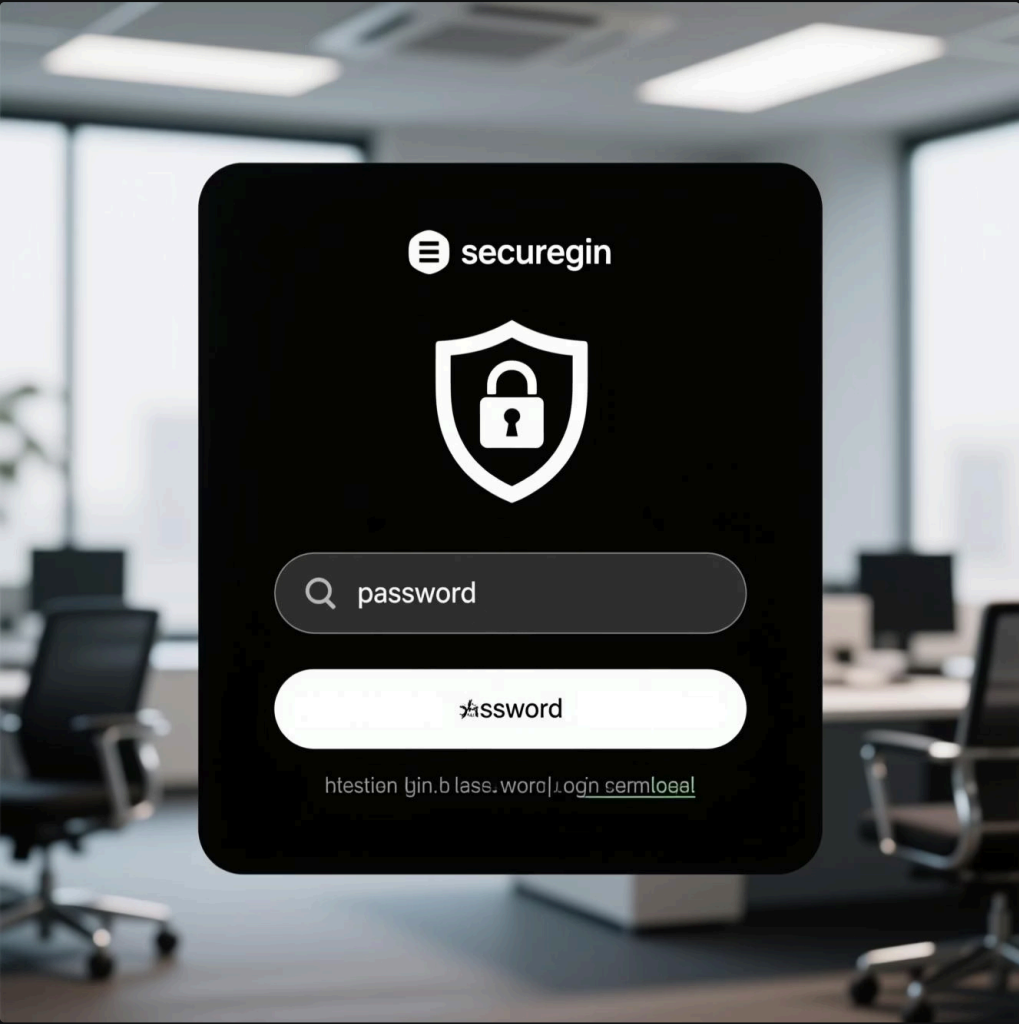
Em um sistema de login, a senha do usuário é um dado sensível que precisa ser protegido.

Solução com Visibilidade:

- `private $senha`: Impede acesso direto à senha
- Método público para autenticação
- Método privado para validar credenciais

Benefícios:

Proteção contra acesso não autorizado e centralização da lógica de validação de senhas.



```
class Usuario {
    public $email;
    private $senha;
    private $tentativasLogin = 0;

    public function autenticar($senhaDigitada) {
        if ($this->validarSenha($senhaDigitada)) {
            $this->tentativasLogin = 0;
            return true;
        } else {
            $this->tentativasLogin++;
            return false;
        }
    }

    private function validarSenha($senhaDigitada) {
        // Verificação segura
        return password_verify(
            $senhaDigitada,
            $this->senha
        );
    }
}
```

Cenário Real: Conexão com Banco de Dados

```
class Conexao {
    private $host = "localhost";
    private $usuario = "root";
    private $senha = "senha_bd";
    private $banco = "meu_app";
    private $conexao;

    public function __construct() {
        $this->conectar();
    }

    private function conectar() {
        try {
            $this->conexao = new PDO(
                "mysql:host={$this->host};dbname={$this->banco}",
                $this->usuario,
                $this->senha
            );
            $this->conexao->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch (PDOException $e) {
            die("Erro na conexão: " . $e->getMessage());
        }
    }

    public function executarQuery($sql, $params = []) {
        try {
            $stmt = $this->preparar($sql);
            $this->bindParams($stmt, $params);
            $stmt->execute();
            return $stmt;
        } catch (PDOException $e) {
            die("Erro na execução: " . $e->getMessage());
        }
    }

    private function preparar($sql) {
        return $this->conexao->prepare($sql);
    }

    private function bindParams($stmt, $params) {
        foreach ($params as $key => $value) {
            $stmt->bindValue($key, $value);
        }
    }
}
```

Neste exemplo, `host`, `usuario`, `senha` e métodos internos são **privados** para proteger dados sensíveis de conexão.

Cenário Real: Hierarquia de Classes

Exemplo de Hierarquia

Em um sistema de e-commerce, precisamos de diferentes tipos de produtos com características comuns, mas comportamentos específicos.



```
class Produto {
    protected $id;
    protected $nome;
    protected $preco;
    protected $estoque;

    public function __construct($id, $nome, $preco, $estoque) {
        $this->id = $id;
        $this->nome = $nome;
        $this->preco = $preco;
        $this->estoque = $estoque;
    }

    public function exibirDetalhes() {
        return "{$this->nome} - R$ {$this->preco}";
    }

    protected function atualizarEstoque($quantidade) {
        $this->estoque += $quantidade;
    }
}

class ProdutoFisico extends Produto {
    protected $peso;
    protected $dimensoes;

    public function __construct($id, $nome, $preco, $estoque, $peso, $dimensoes) {
        parent::__construct($id, $nome, $preco, $estoque);
        $this->peso = $peso;
        $this->dimensoes = $dimensoes;
    }

    public function calcularFrete() {
        // Usa o atributo protegido $peso
        return $this->peso * 10;
    }
}

class ProdutoDigital extends Produto {
    protected $tamanhoArquivo;
    protected $formato;

    public function gerarLinkDownload() {
        // Acessa atributos protegidos da classe pai
        return "download/{$this->id}/{$this->nome}.{$this->formato}";
    }
}
```

Atributos `protected` permitem que as subclasses acessem e modifiquem esses dados, mantendo-os protegidos de acesso externo.

Importância do Encapsulamento

Segurança

Protege dados sensíveis e implementações internas contra acessos indevidos.

Evita que o estado interno do objeto seja alterado de maneira inconsistente.

Reuso de Código

Facilita a manutenção e evolução da base de código.

Permite modificar a implementação interna sem afetar o código cliente.

Clareza e Organização

Define interfaces claras para interação entre objetos.

Esconde a complexidade desnecessária e reduz o acoplamento.

A visibilidade é uma ferramenta fundamental para implementar o encapsulamento, permitindo que você controle precisamente o que é exposto e o que é protegido em suas classes.



Boas Práticas de Visibilidade

Princípio do Menor Privilégio

- Sempre use a **visibilidade mais restritiva possível**
- Declare atributos como `private` por padrão
- Só altere para `protected` se houver necessidade real de acesso por subclasses
- Evite atributos `public` - use getters e setters

Interfaces Claras

- Exponha apenas o que é necessário para uso externo
- Utilize métodos `public` como "contratos" da classe
- Agrupe funcionalidades relacionadas em classes coesas



"Esconda tudo o que for possível. Exponha somente o que for absolutamente necessário."

Getters e Setters

Métodos **getters** e **setters** permitem acessar e modificar atributos privados de forma controlada, mantendo o encapsulamento.

```
class Produto {
    private $nome;
    private $preco;
    private $estoque;

    public function __construct($nome, $preco, $estoque) {
        $this->nome = $nome;
        $this->setPreco($preco);
        $this->setEstoque($estoque);
    }

    // Getters - acessam valores
    public function getNome() {
        return $this->nome;
    }

    public function getPreco() {
        return $this->preco;
    }

    public function getEstoque() {
        return $this->estoque;
    }

    // Setters - modificam valores com validação
    public function setNome($nome) {
        if (strlen($nome) >= 3) {
            $this->nome = $nome;
        } else {
            throw new Exception("Nome deve ter pelo menos 3 caracteres");
        }
    }

    public function setPreco($preco) {
        if ($preco > 0) {
            $this->preco = $preco;
        } else {
            throw new Exception("Preço deve ser maior que zero");
        }
    }

    public function setEstoque($estoque) {
        if ($estoque >= 0) {
            $this->estoque = $estoque;
        } else {
            throw new Exception("Estoque não pode ser negativo");
        }
    }
}
```

Vantagens: validação centralizada, controle sobre modificações, possibilidade de depuração e facilidade para adicionar lógica de negócio.

Erros Comuns de Visibilidade

Expor Atributos Sensíveis

Problema: Definir atributos críticos como `public` compromete a segurança e permite modificações sem validação.

Exemplo:

```
class Usuario {  
    public $senha; // ERRADO!  
}
```

Solução: Declare atributos como `private` e use getters/setters para controlar o acesso.

Métodos Públicos Desnecessários

Problema: Expor métodos internos que deveriam ser privados aumenta o acoplamento e fragiliza o código.

Exemplo:

```
class Processador {  
    public function validarDados() { // Deveria ser private  
        // Lógica interna  
    }  
}
```

Solução: Mantenha métodos auxiliares como `private` ou `protected`.

Uso Incorreto de Protected

Problema: Usar `protected` quando não há necessidade real de acesso por subclasses aumenta o acoplamento na hierarquia.

Exemplo:

```
class Base {  
    protected $config; // Deveria ser private se não usado por subclasses  
}
```

Solução: Use `protected` apenas quando for realmente necessário para herança.

Conclusão

Visibilidade = Controle

Os modificadores de acesso permitem controlar precisamente quem pode interagir com atributos e métodos.

Encapsulamento = Proteção

Ocultar implementações internas e expor apenas interfaces bem definidas protege a integridade dos seus objetos.

Prática = Domínio

A aplicação correta dos conceitos de visibilidade vem com a prática e análise de casos reais.

Os conceitos aprendidos hoje são fundamentais para desenvolver sistemas orientados a objetos robustos, seguros e bem estruturados. Continuem praticando!

Próxima Aula: Construtores e Destrutores em PHP



Resolução em Sala

Dinâmica de Trabalho

Vamos dedicar este tempo para trabalhar na lista de exercícios A1/1 relacionada à visibilidade em PHP.

- **Trabalho individual:** Cada aluno deve desenvolver sua própria solução
- **Suporte do professor:** Tire dúvidas específicas sobre os conceitos ou implementação
- **Discussão em grupo:** Compartilhe abordagens e soluções com os colegas

📄 Lembre-se de testar seu código com diferentes valores para garantir que as validações de visibilidade estejam funcionando corretamente.



Passo a Passo Sugerido

1. Leia atentamente cada exercício
2. Projete a estrutura das classes no papel
3. Implemente os atributos com visibilidades adequadas
4. Desenvolva os métodos getters e setters
5. Adicione a lógica de validação
6. Teste seu código com diferentes cenários
7. Refine sua implementação conforme necessário

Exercícios da Aula - Lista A1/1

 Todos os exercícios devem ser realizados individualmente e enviados através do Portal de Ensino da Unidade I até as 22h30.

1. Crie uma classe `Produto` com os atributos `nome` e `preco`. Ambos devem ser públicos. Instancie um objeto e exiba seus valores.
2. Modifique a classe `Produto` tornando `preco` privado. Crie métodos `getPreco` e `setPreco` para manipular o valor.
3. Crie uma classe `ContaBancaria` com atributo privado `saldo`. Implemente métodos para depositar e sacar.
4. Crie uma classe `Funcionario` com atributo protegido `salario`. Crie uma subclasse `Gerente` que permita acessar e modificar este valor.
5. Implemente uma classe `Usuario` com atributo privado `senha`. Crie método `verificarSenha($senhaDigitada)` que retorna `true/false`.
6. Crie uma classe `Config` com atributo protegido `parametros`. Crie subclasses que leiam e modifiquem esses parâmetros.
7. Desenvolva uma classe `Pedido` com atributo privado `itens` (array). Adicione métodos para inserir e listar itens.
8. Crie uma classe `Cliente` com nome público, CPF protegido e telefone privado. Teste acessos dentro e fora da classe.
9. Refatore a classe `ContaBancaria` para permitir saque apenas se o saldo for suficiente.
10. Projete uma classe `ConexaoBD` com método privado `conectar()`. Exponha apenas um método público `getConexao()` para retornar a conexão.



Crerios de Avaliao

- Aplicao correta dos nveis de visibilidade
- Implementao de getters e setters adequados
- Validao de dados nos setters
- Organizao e clareza do cdigo
- Justificativa das escolhas de design